Gruppenarbeit

Docs -> Ort der Kommunikation für Stakeholder/ zu7m Informieren

Docs: Wiki, README and Pages

README

Was ist das?

Die zentrale Einstiegsseite eines Repositories.

Was macht es?

Erklärt auf einen Blick:

- worum es im Projekt geht
- was das Projekt kann
- wie man es benutzt
- welche Voraussetzungen bestehen

Wofür ist es gut?

- **Erster Eindruck für neue Nutzer oder Entwickler**
- Schnellverständnis des Projekts
- Technische Visitenkarte deines Repos

Typisch: Projektbeschreibung, Features, Installation, Beispiele, Lizenz.
https://docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme
https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes

---

Wiki

Was ist das?

Ein separates Dokumentations-System pro Repository.

Was macht es?

Ermöglicht **umfangreiche, strukturierte Doku** mit mehreren Seiten:

- Architektur
- Konzepte
- API-Details

- interne Abläufe

Wofür ist es gut?

- Tiefergehende technische Dokumentation
- Wissensspeicher für Teams
- Ideal für Dinge, die zu lang für README sind

Funktional eher wie ein internes Mini-Handbuch.
https://docs.github.com/en/communities/documenting-your-project-with-wikis/adding-or-editing-wiki-pages

---

Pages

Was ist das?

Eine statische Website direkt aus deinem Repository.

Was macht es?

Veröffentlicht Inhalte öffentlich als Website:

- Produktseiten
- Projektdokumentation
- Demo-Seiten

Wofür ist es gut?

- **Öffentliche Präsentation deines Projekts**
- Nutzerfreundliche Doku
- Marketing + Technik kombiniert

Im Gegensatz zu README/Wiki ist Pages klar *nach außen* gerichtet.
https://docs.github.com/de/pages/getting-started-with-github-pages/creating-a-github-pages-site

Automatisierte Doku (z. B. MkDocs + Actions)

Kerngedanke:

Dokumentation wird wie Code behandelt:

- versioniert
- automatisch gebaut
- automatisch veröffentlicht

Nutzen:

- Keine veraltete Doku
- jede Version hat passende Dokumentation
- Änderungen laufen über Pull Requests

Das ist klassisches **Docs-as-Code**.

Releases & Change Management

Knowledge Management (Wissensmanagement)

Bedeutung hier:

GitHub wird zur **zentralen Wissensquelle**:

- alles an einem Ort
- durchsuchbar
- Community kann beitragen

Sehr wichtig für größere Projekte oder Open Source.

Version Management

Was ist das?

Verwaltung von Software-Versionen.

Beispiel:

- 2.0.0 → Breaking Changes
- 2.1.0 → Neues Feature
- 2.1.1 → Bugfix

Tags vs Releases

- **Tag** = technischer Marker im Git
- **Release** = offizieller Meilenstein mit Beschreibung + Artefakten

Nutzen:

- klare Zustände deiner Software
- reproduzierbare Builds
- Nutzer wissen, was stabil ist

Change Control

Was bedeutet das?

Kontrolle über Änderungen.

Beinhaltet:

- automatische Changelogs
- geplante Releases
- Kommunikation an Stakeholder

Wofür gut?

- Niemand wird von Änderungen überrascht
- Features sind nachvollziehbar
- Unternehmen behalten Planungssicherheit

Sehr wichtig im professionellen Umfeld.

Deployment Strategies

Das beschreibt *wie* neue Versionen ausgerollt werden.

Beispiele:

▸ Phased Deployment

Nicht alle Nutzer gleichzeitig → Risiko minimieren.

▸ Feature Flags

Code ist schon live, Feature aber deaktiviert.
Aktivierung später per Schalter.

▸ Rollback

Wenn etwas kaputt geht → sofort zurück zur alten Version.

https://github.com/mkdocs
https://github.blog/changelog/
https://github.com/marketplace/actions/release-scheduler
https://github.com/eslint/eslint/issues/17856

https://docs.github.com/en/communities/documenting-your-project-with-wikis/adding-or-editing-wiki-pages
https://docs.github.com/de/pages/getting-started-with-github-pages/creating-a-github-pages-site
https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
https://docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme